

Version control with git

or how to write code without going crazy

January 16, 2026

Agenda – Day Two

- Recap
 - Why git?
 - Basic workflow?
 - Questions?
- Advanced Topics
 - Branching
 - Rebasing
 - Tagging
 - Licenses

Recap

Recap

Why Git?

- Keep track of your work
- Restore previous versions
- Enable collaboration

Recap

Basic Workflow?

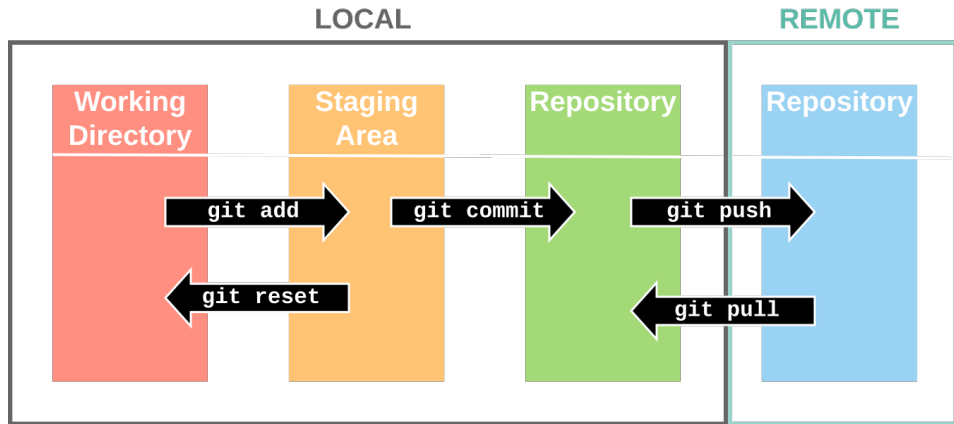


Figure 1: Basic operations

Recap

Basic Workflow?

- Saving work
 - add
 - commit
 - push
- Downloading
 - clone (initially)
 - pull (afterwards)

Advanced Topics

Advanced Topics

Branching

- Problem: Broken intermediary states
- Solution: Feature branches (nice tutorial at [git-scm](#))
- Create branch: `git checkout -b branchname` (if it doesn't exist)
- Commit the branch: `git commit -m "message"`
- Push the branch: `git push -u origin branchname`
- Merge two branches (here branchname with master):
 - `git checkout master`
 - `git merge branchname` (In case of conflict: Edit, commit)

Advanced Topics

Network Graph – Branching

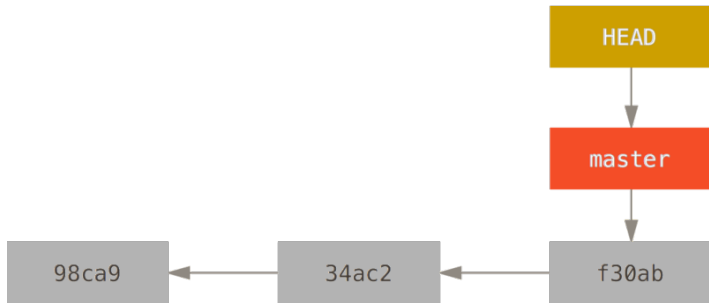


Figure 2: Before branching

Advanced Topics

Network Graph – Branching

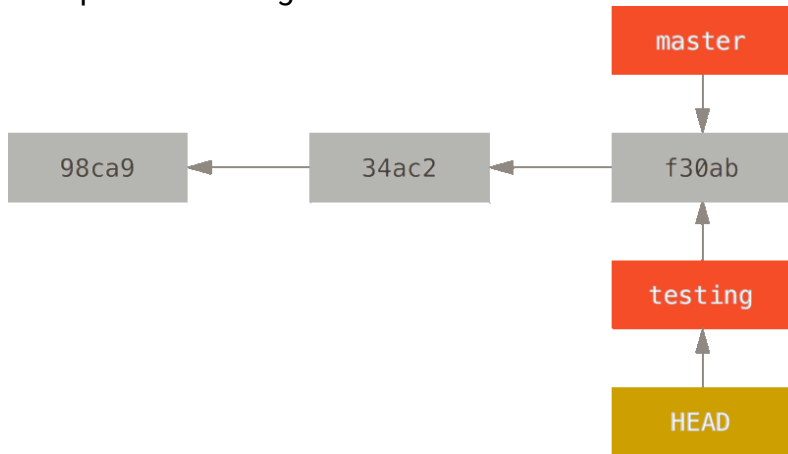


Figure 3: After `git checkout -b testing`

Advanced Topics

Network Graph – Branching

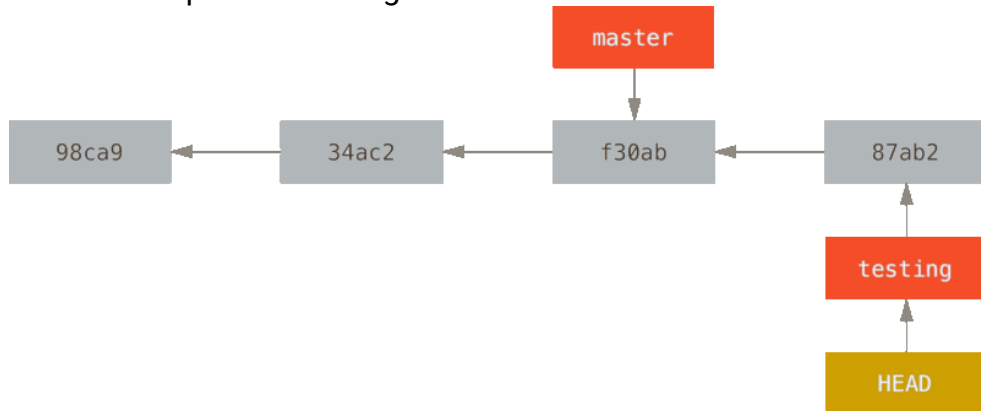


Figure 4: After `git commit -m "testing commit"`

Advanced Topics

Network Graph – Branching

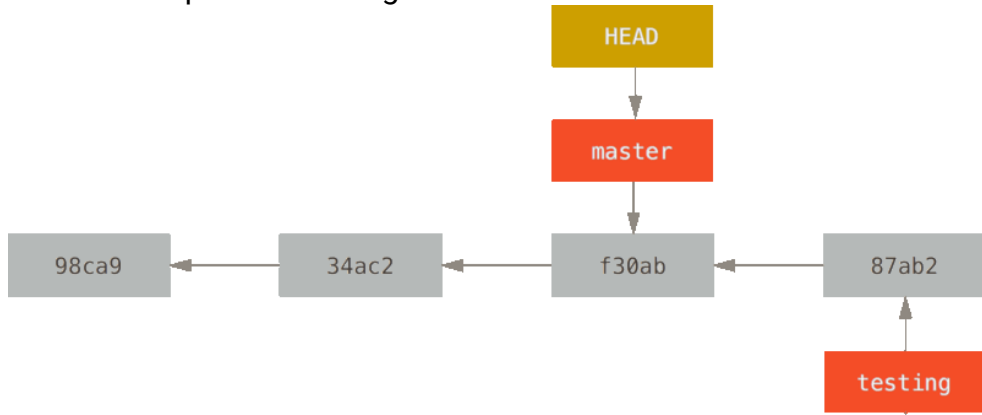


Figure 5: After `git checkout master`

Advanced Topics

Network Graph – Branching

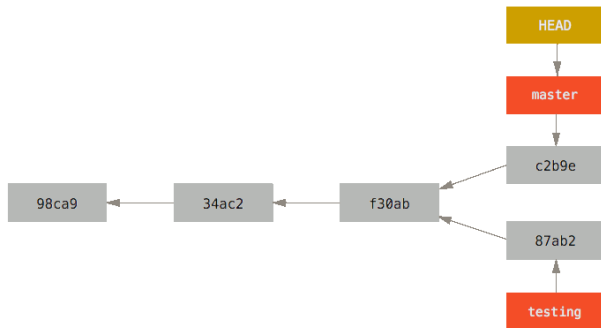


Figure 6: After `git commit -m "master commit"`

Advanced Topics

Network Graph – Branching

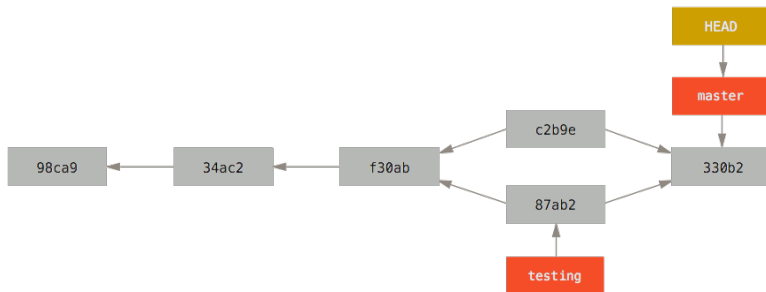


Figure 7: After git merge testing

Advanced Topics

Exercise: Feature branch

- New feature branch for Markdown-Cheatsheet
- Write a few notes about Markdown syntax in a new markdown file
- Have at least two commits on the branch (small changes, commit intermediate states)
- Push each commit, then look at the network graph
- Merge to master, push, look at the network graph

Advanced Topics

README

- Default file shown on repo landing page
- Main information source for users
 - What is the tool/repo for?
 - How is it supposed to be used? - How to install/run it - Example call with example data - Expected output with explanation - What can go wrong?

LIKELIHOOD YOU WILL GET CODE WORKING
BASED ON HOW YOU'RE SUPPOSED TO INSTALL IT:



Figure 8: XKCD #1742

Advanced Topics

Tags

- Generally: Link to a specific commit (see [git-scm](#))
- Contain metadata like date and comment (except lightweight tags)
- Creation: `git tag -a v0.1 -m "The first version"`
- Show info: `git show v0.1`
- push: `git push origin v0.1` or `git push origin --tags`
- Checkout:
 - Only look, no touch: `git checkout v0.1` (detached HEAD)
 - Change (new branch): `git checkout -b branch_v0.1 v0.1`

Advanced Topics

Semantic versioning

- Convention for versioning of software
- Clear information on effects of changes and compatibility
- No defined standard, just a convention



Figure 9: Meaning of SemVer fields

Advanced Topics

Exercise: Tagging

- Two changes (in two separate feature branches):
 - Add README.md
 - Update git cheatsheet
- merge with master
- tag as "v0.1"

Advanced Topics

Releases

- Stable “snapshots” of the repositories state
- Can be enriched with further data/files (e.g., binaries)
- Simple mechanism for distributing software
- Based on tags
- .zip and .tar.gz of repository are automatically generated
- Described in depth in the [GitHub documentation](#)

Advanced Topics

Exercise: Release

- Build PDFs from your cheatsheets
- Create a release from the v1.0 tag
- Include the PDFs in the release

Advanced Topics

Licenses - Basics

- Disclaimer: IANAL
- Important: Do set a license! It ensures:
 - Re-usability
 - Own rights
 - Self-protection, e.g., liability (don't feed the lawyer)
- Basic differences:
 - Open Source vs. Closed Source
 - Copyright vs. Copyleft

Advanced Topics

Licenses - Short How-To

- **LICENSE** in repository root
 - GitLab, GitHub: Templates available for typical licenses
 - Look at it together – LICENSE in example repository
- Attention: Licenses can be incompatible!
 - Caution generally required when: - Modifying code (compatible with own license?) - Redistribution (requirements, e.g., making source available)
 - Simple use without modification usually unproblematic

Licenses - Cheat Sheet

- Overview: <https://tldrlegal.com/>, <https://choosealicense.com/>
- Compatibility chart by [David A. Wheeler](#)

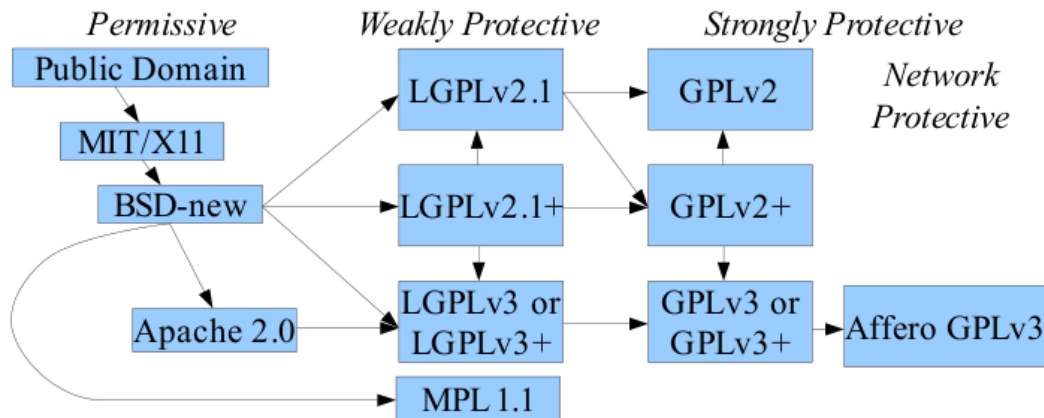


Figure 10: Lizenzcompatibility

Citations

- Makes citing your work easier
- Add a file CITATION.cff in [citation file format](#)
 - In GitHub: “Cite this repository” Automatically displayed in sidebar
 - Standard: Software citation
 - Once paper is published: set preferred-citation to article and wait!
- Example: [GitHub-Dokumentationsseite](#)

Exercise: Citability

- Finalise the repository:
 - Set a license
 - Add last notes
 - Add new tag
 - Register the repository with [Zenodo](#)
 - Create another release on GitHub
 - Add a CITATION.cff with the DOI from Zenodo

More practice: Oh my git

- [Game](#) for practicing and visualizing git concepts
 - Real git, interaction via commandline or playing cards
 - Visualization of the repository structure (like “network” in GitHub)
 - Separate “missions” for different concepts

Image sources

Folie 6: [“notFinal.doc”](#), Jorge Cham, “Piled Higher and Deeper”, PHD comics

Folie 8: [“Git: Reference Sheet”](#), New Zealand eScience Infrastructure

Folie 9: [“Deltas”](#), Scott Chacon und Ben Straub, Pro Git book

Folie 16: [“Git”](#), Randall Munroe, XKCD Folien 17, 19, 22, 25, 32-37: [“Git Branching”](#), Scott Chacon und Ben Straub, Pro Git book

Folie 33: [“Will it work”](#), Randall Munroe, XKCD

Folie 35: [“Version numbers in Semantic Versioning”](#), Surjit Bains, Wikipedia

Folie 41: [“FLOSS License chart”](#), David A. Wheeler

Folie 44: [“Oh my git! logo”](#), bleeptrack & blinry